Cross-entropy loss

$$f(x_i, W) = Wx_i$$ (score function)

$$L_i = -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right) \qquad L_i = -f_{y_i} + \log\sum_j e^{f_j}$$

We call L_i *cross-entropy loss*

$P(y_i \mid x_i; W)$

i.e. we're minimizing the negative log likelihood.

Losses

- Cross-entropy loss is just one possible loss function
  - One nice property is that it reinterprets scores as probabilities, which have a natural meaning

- SVM (max-margin) loss functions also used to be popular
  - But currently, cross-entropy is the most common classification loss

The story so far: linear classification

- What we have: a score function and loss function
  - Score function maps an input data instance (e.g., an image) to a vector of scores, one for each category
  - Last time, our score function is based on linear classifier

$$f(x, W) = Wx + b$$

f: score function
x: input instance
W, b: parameters of a linear (actually affine) function

- Find **W** and **b** that minimize a *loss* over labeled training data, e.g. cross-entropy loss

$$L = \frac{1}{N} \sum_i -\log \left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}} \right)$$

Questions?

Neural networks

(**Before**) Linear score function: $f = Wx$

Slide credit: Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks

(**Before**) Linear score function: $f = Wx$

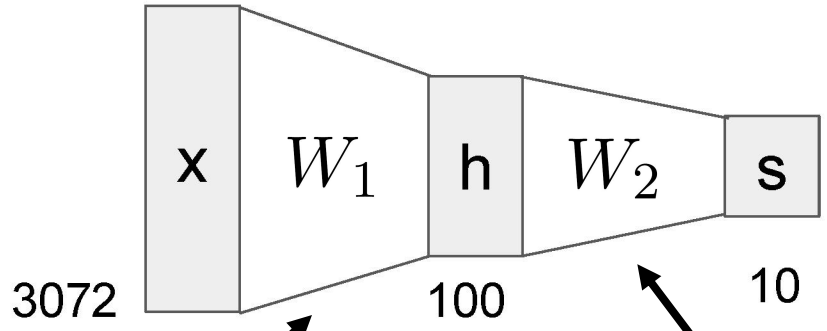
(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

Slide credit: Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



W_1
(100 x 3072 matrix)

h
100D intermediate vector

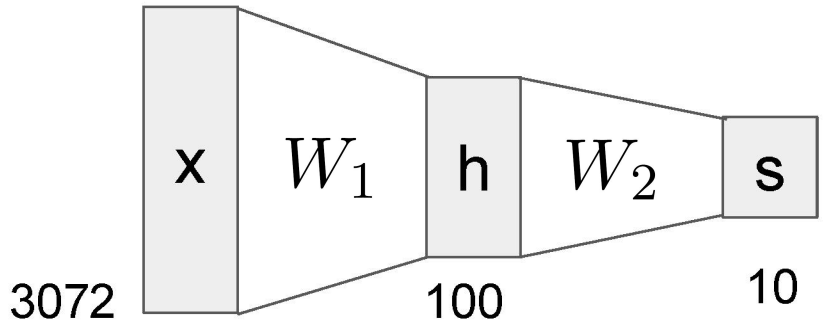
W_2
(10 x 100 matrix)

Slide credit: Fei-Fei Li & Andrej Karpathy & Serena Leung

Neural networks

(**Before**) Linear score function: $f = Wx$

(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



- Total number of weights to learn:

  3,072 x 100 + 100 x 10 = 308,200